

TECHNICAL DEEP DIVE

32 张 H100 承载 128K MoE RL：显存解耦、通信重叠与配置收敛

基于技术演讲与配套材料整理

配套材料：让长序列MoE_RL训练更好调.pptx

2026-08-03

以 Qwen3.5-35B-A3B 为例，拆解一条从 127.53 TFLOPS/GPU 基线推进到接近 190 TFLOPS/GPU 的组合优化路径。

强化学习训练需要频繁调整 reward、rollout、数据配方和超参数。如果每组实验都要重新盲扫并行配置，系统试错成本很可能先于算法验证成本失控。

本文讨论一个边界明确的案例：在 32 张 H100 上，让 Qwen3.5-35B-A3B 处理 128K 全局序列。重点不是寻找任意集群规模下的峰值配置，而是解释如何依次处理动态激活、完整 logits、静态模型状态和 MoE 通信，并将高度耦合的并行搜索收敛为一套可复用的训练 recipe。

文中的实验数字、架构描述和性能结论均来自演讲 PPT 与转写材料。涉及设计动机而缺少受控实验时，本文会明确标注证据边界。

适读人群与前置知识

本文面向具备分布式训练、Transformer 和 GPU 性能基础，但尚不熟悉长序列 MoE RL 配置与 Megatron 工程实现的工程师。

阅读前最好已经了解：

- 数据并行、张量并行和流水线并行的基本概念；
- Transformer 前向、反向、activation 与交叉熵计算；
- MoE 的 expert routing、dispatch、expert GEMM 和 combine 流程；
- 静态模型状态、动态激活显存、计算时间和通信时间的区别。

不要求预先掌握 Megatron、FSDP2 或 chunked EP 的具体实现。

读完本文，应当能够：

- 解释 PP、TP、EP、CP、重计算与 local sequence length 的耦合关系；
- 区分 full recompute、Linear CE、FSDP2 和 chunked EP 各自处理的瓶颈；
- 正确解读 critical-rank trace、单层代理实验和完整组合路径；
- 把 PP、TP、EP、CP 的四维盲扫压缩为少量约束驱动的策略；
- 理解 primitive 化为什么能缩小优化接入与验证范围。

一、先定义问题：RL 需要的不是另一套预训练配置

首先要回答：为什么这个案例需要单独设计 RL recipe，而不是直接复用某套预训练参数？

MoE (Mixture of Experts, 混合专家) 是一种稀疏模型结构。token 经路由后被分发到不同专家，完成专家计算，再将结果合并。它在常规 Transformer 计算之外增加了路由、跨设备 dispatch/combine 和专家负载均衡问题。

RL recipe 则不只是一个配置文件，而是一组可以稳定启动、能够被后续实验复用的训练选择，包括资源规模、并行组合、显存策略和运行参数。

工作负载差异先于并行参数

预训练与 RL 的工程重心可以概括为：

工作负载	典型资源组织	主要工程目标	需要探索的变量
预训练	大型单作业长期运行	维持稳定吞吐	仍需调参，但运行阶段更强调持续利用资源
RL	将有限资源分给多组实验	多实验并行、快速试错、降低启动门槛	reward、rollout、数据配方和超参数

这只是工作负载倾向，并不表示预训练不需要试验，也不表示 RL 不关心吞吐。真正的差异在于，RL 往往同时探索算法、数据和系统配置。若每个候选实验都要先进行一轮漫长的并行组合搜索，GPU 时间就会大量消耗在“如何启动”上。

三个数字共同限定问题

为什么要看下面这张图？因为模型、硬件和序列长度必须放在一起，才能定义本文所说的“可运行”。



图 1：案例由模型、GPU 数量和全局序列长度三项边界共同界定。来源：演讲 PPT，第 5 页。

图中的三张指标卡分别给出：

- **Qwen3.5-35B-A3B MoE**：指定训练对象。该页没有进一步拆分总参数与激活参数口径，不能仅凭名称推算单卡状态占用。
- **32 张 H100**：限定可用资源。模型状态、长序列激活和 MoE 通信都必须在这一规模内解决。
- **128K global sequence length**：限定全局序列口径。它不等于单条 prompt、单次 rollout 或单张 GPU 上的局部序列长度。

该页也没有说明 H100 的显存版本、训练精度和 batch size。因此，32 张 H100 与 128K 只能作为本案例的边界，不能解释为适用于任意 MoE 模型的容量结论。

三项约束形成了一条直接的工程因果链：

RL 需要并行探索多组方案

→ 单组实验可使用的资源有限

- 长序列与 MoE 同时增加显存和通信压力
- 每次重新搜索系统配置都会拖慢试验
- 因而需要稳定、可复用的运行 recipe。

例如，团队准备比较两种 reward、两种 rollout 配方和两组超参数时，即使不继续扩展组合，也要启动多次训练。若每次都重新盲扫并行配置，一些组合会直接 OOM，另一些虽然能够运行，却可能受限于通信或调度。此时最先失控的往往不是算法复杂度，而是试错成本。

所以，本文要解决的核心问题是：如何在固定的 32 张 H100 上，为 Qwen3.5-35B-A3B 找到能够承载 128K 全局序列、稳定启动并可供后续 RL 实验复用的运行点。

二、基线为什么难调：六个观察量投影成五类压力

本节要回答的不是“哪个并行度越大越好”，而是当前配置被哪类资源卡住，以及应当由哪种机制承担这部分压力。

需要同时考虑：

- PP: Pipeline Parallelism, 流水线并行；
- TP: Tensor Parallelism, 张量并行；
- EP: Expert Parallelism, 专家并行；
- CP: Context Parallelism, 上下文并行；
- activation 重计算策略；
- local sequence length, 即单张 GPU 实际承担的局部序列长度。

其中 local seq 是其他选择共同作用的结果，并非完全独立的旋钮。

用五类压力建立统一坐标

为什么要看下面这张矩阵？因为它把不同并行策略投影到同一组资源维度，避免只盯着显存判断配置。

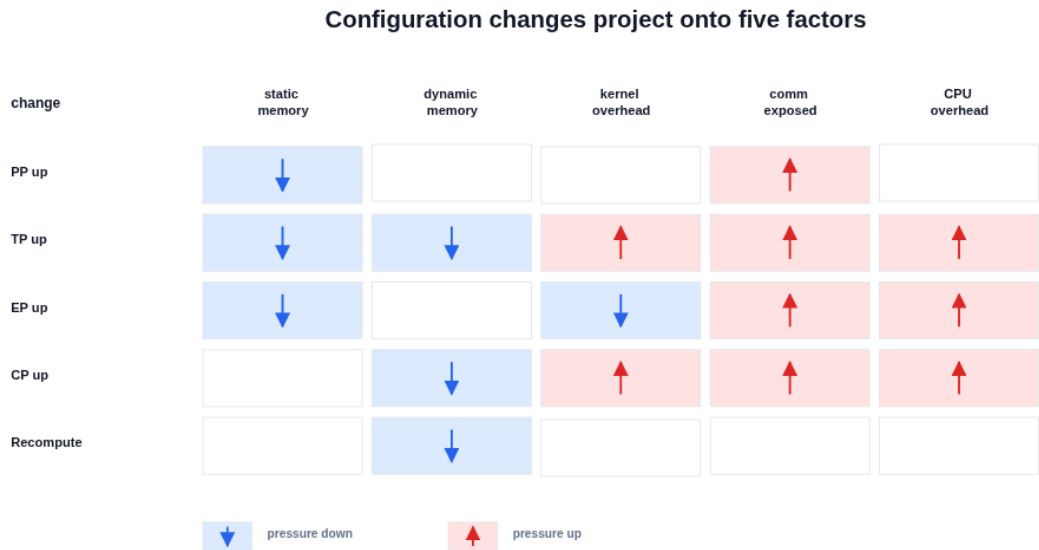


图 2：箭头表示压力变化方向；空白项不代表影响为零。来源：演讲 PPT，第 7 页。

五类压力分别是：

- **静态显存**：参数、梯度和优化器状态等模型状态占用；
- **动态显存**：随序列、批量和中间激活变化的占用；
- **kernel 开销**：切分后计算变碎或矩阵形状变差造成的效率损失；
- **暴露通信**：无法被计算掩盖、直接进入关键路径的通信时间；
- **CPU 开销**：kernel launch、通信调度等主机侧压力。

沿矩阵逐项看：

增大

会把模型切到更多流水阶段，主要降低每个阶段承担的静态状态，但也会增加流水线空泡。PP 对动态显存没有同等直接的削减作用，因为流水调度可能要求部分 stage 同时保留多组 microbatch 的激活。

增大 TP

可以同时降低静态与动态显存，但也会把矩阵切薄，可能导致 kernel 形状变差；collective 通信随之增加，较短、较碎的计算还会让 CPU 调度更容易暴露。

增大 EP

会减少每个 rank 持有的 expert 数量，从而降低静态显存，并改变 grouped GEMM 的工作规模；代价是 dispatch 和 combine 需要更多 all-to-all，暴露通信与 CPU 压力会上升。转写还提到 expert imbalance 可能推高动态显存，但 PPT 没有量化这一关系。

增大 CP

直接切分序列，主要缓解动态显存；与此同时，局部计算变短，kernel、通信和 CPU 调度压力可能上升。

启用重计算

以重复前向计算换取更少的 activation 保留。材料明确支持的直接收益是动态显存下降，而不是静态显存下降。

这些箭头只表示影响方向，没有绑定具体起止配置，也不代表影响幅度在不同模型和硬件上保持不变。

local seq 为什么会放大耦合

演讲采用了一条简化关系。设全局序列长度为 S，则：

$$S_{local} = (S) / (TP \times CP)$$

例如，当 S=128K、TP=2、CP=2 时：

$$S_{local} = (128K) / (2 \times 2) = 32K$$

若把 CP 从 2 提高到 4，local seq 会进一步缩短到 16K。这样通常有利于动态显存，却意味着单次本地计算持续时间更短，kernel launch、通信和 CPU 调度更难被足够长的计算区间掩盖。

因此，local seq 既是 TP、CP 的结果，又会反过来影响 kernel 形状和通信暴露程度。PP 与 EP 虽然没有出现在这条简化公式中，却会改变 stage 划分、专家计算和通信拓扑；重计算则改变需要保存的 activation。

最终的因果链是：

并行与重计算配置

→ 每卡模型状态和激活规模

→ local seq 与 kernel 形状

→ 通信能否被计算覆盖

→ CPU 调度是否暴露。

这只是演讲采用的简化关系，不应扩展为所有序列并行实现的统一公式。后文 logits 算例里的 local_tokens 也应按对应页面口径单独理解，不能与这里的 local seq 不加区分地混用。

配置矩阵中的现实后果

为什么要看下面这张配置矩阵？因为它展示了耦合的直接后果：大量组合并不是“稍慢”，而是无法运行。

35B Megatron baseline CSV

		TP * CP			
		TP2 * CP1	TP2 * CP2	TP2 * CP4	TP2 * CP8
PP * EP	PP4 * EP8	full	OOM	127.5 TFLOPS 42.9 GB	
		selective		OOM	
	PP2 * EP16	full	OOM	81.1 TFLOPS 38.2 GB	
		selective		OOM	76.9 TFLOPS 57.6 GB

图 3：横轴为 TP×CP，纵轴按 PP×EP 和重计算策略分组；红色单元格表示 OOM。来源：演讲 PPT，第 9 页。

在 35B Megatron baseline 中，`PP4×EP8 + full recompute + TP2×CP2` 是一个可运行点，页面记录为：

- 127.5 TFLOPS/GPU；
- 42.9 GB 峰值显存。

它周围存在多个 OOM 单元格。但不同单元格往往同时改变多项并行参数，因此不能据此做单变量归因。空白单元格也只能理解为页面没有给出结果，不能自行补写为 OOM 或未测试。

该页没有完整列出 TFLOPS 统计口径、GPU 数量、序列长度、批量大小和显存容量上限。因此，它能够证明的是搜索空间高度耦合、存在大量 OOM，而不是某个并行维度具有固定收益。

PP 还有额外的调度边界。问答中的示例指出：若 `PP=8`，microbatch 数量也只有 8，pipeline bubble 约为 50%，计算时间与空泡时间大致相当。这个数字只有转写证据，并且只适用于该示例，不能推广为 `PP=8` 的固定效率。

基线给出的第一条经验由此明确：先拆资源职责，再比较吞吐。动态 activation 与 local seq 的矛盾最先需要处理，而这会引出一项看似反直觉的选择 - 接受完整重计算。

三、第一步反直觉选择：用完整重计算换取更大 local seq

本节要回答：明知会增加计算量，为什么仍然选择 full recompute？

先区分两种策略：

- **full recompute (完整重计算)**：每层前向结束后丢弃对应 activation，反向传播到该层时重新执行所需计算；

- **selective recompute (选择性重计算)**：只重算部分操作，其余 activation 继续保留。

材料没有给出 selective recompute 的完整覆盖范围，因此本文不推断具体算子清单。

为什么“少重算”可能带来连锁压力

不使用 full recompute 可以节省重算 FLOPs，却需要保留更多 activation。在长序列场景中，这可能形成如下链路：

保留 activation

→ 动态显存持续承压

→ 必须缩短 local seq

→ 增大 CP

→ 本地计算更加碎片化

→ kernel 与 CPU launch 开销更容易暴露

→ CUDA Graph 也被带入联动调优范围。

CUDA Graph 是通过记录并回放 GPU 调度来降低 CPU kernel launch 开销的机制。在这条路径中，它不是一个孤立开关，而是由“显存不足 - 增加切分 - 计算碎片化”逐步推到前台的优化项。

不过，材料没有提供这条路径的同配置吞吐、显存和延迟对照。因此，上述内容是设计权衡链，而不是量化结论。

Path B：先释放 activation，再拉长本地计算窗口

为什么要看下面这张图？因为它同时画出了 full recompute 的收益链和必须支付的代价。

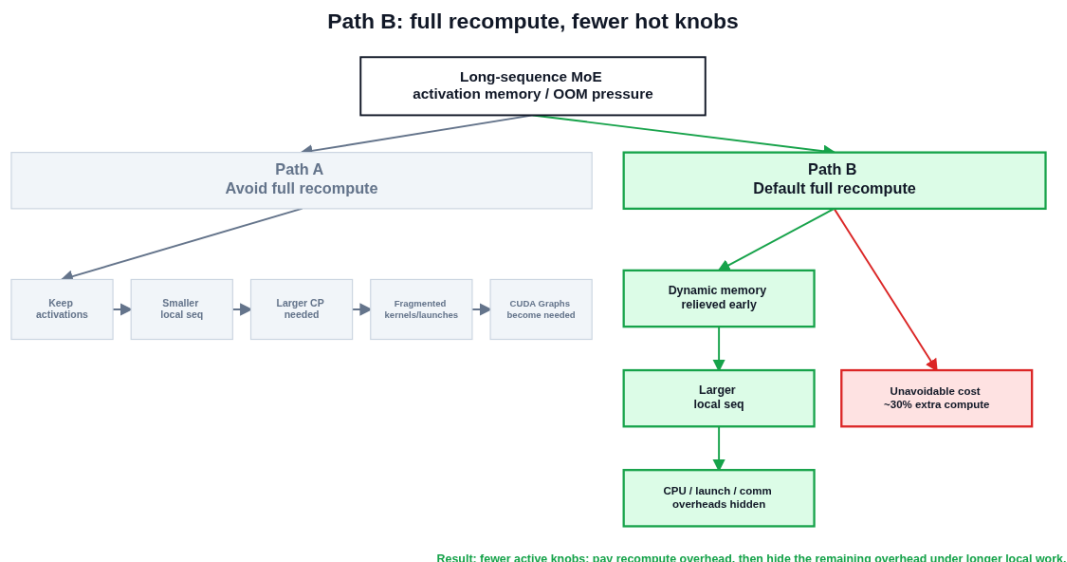


图 4：Path B 以额外计算换取更早释放动态显存和更大的 local seq。来源：演讲 PPT，第 11 页。

图中的收益路径包含三个关键节点：

- 1 **更早缓解动态显存。** 每层前向结束后不再长期保存 activation。这里释放的是激活显存，不是参数、梯度、优化器状态或 loss 侧的全部占用。
- 2 **允许更大的 local seq。** 动态显存留下空间后，系统可以减少仅为压低 activation 峰值而进行的序列切分，让单卡承担更长的局部序列。
- 3 **提供更长的掩盖窗口。** 本地计算持续时间变长后，CPU 调度、kernel launch 和通信有机会与计算重叠，降低其暴露在关键路径上的比例。

图中的另一条支线是代价：约 **30% 额外计算量**。这是材料给出的近似 FLOPs 成本，不等于端到端训练时间必然增加 30%。如果更大的 local seq 改善了 kernel 工作粒度，并掩盖部分调度与通信开销，额外计算量和最终训练时间之间不会保持简单的一比一关系。

同样，“隐藏开销”不表示开销消失。实际效果仍取决于计算长度、通信时序和可实现的重叠程度。

full recompute 的本质，是用确定的重算成本交换动态显存，再用释放出的空间扩大单卡工作量。但它只直接处理 activation。activation 峰值下降后，完整 fp32 logits 和静态模型状态会成为新的显存约束。

四、连续拆掉两类显存峰值：Linear CE 与 FSDP2

扩大 local seq 后，显存问题会从“保存多少 activation”转移到两类来源：

- loss 计算时可能物化的完整 logits；
- 长期驻留的参数、梯度和优化器状态。

两者的生命周期和形成原因不同，必须分别处理。

Linear CE：避免完整 logits 物化

logits 是交叉熵之前沿词表维展开的输出。设：

- local_tokens：当前 loss 计算口径下的本地 token 数；
- vocab_partition：当前 rank 承担的词表分片大小；
- fp32 每个元素占 4 字节。

若实现生成并保留完整 fp32 logits，则估算显存为：

M_logits = local_tokens × vocab_partition × 4 bytes

为什么要看下面这张图？因为它说明只改变 TP 与 CP 的分配方式，不一定能降低完整 logits 的乘积规模。

Linear CE：logits peak 不能忽略

生成并保留完整 fp32 logits 的估算口径：

local_tokens * vocab_partition * 4 bytes

配置口径	local tokens	vocab partition	fp32 logits
TP=2, CP=2	131,072 / 2 = 65,536	248,320 / 2 = 124,160	32.55 GB
TP=1, CP=4	131,072 / 4 = 32,768	248,320	32.55 GB

代表点

TP=1, PP=2, EP=8, CP=4
full recompute

- linear CE 不生成并保留完整 logits，释放 loss-side peak
- 有了这部分空间，才能把并行布局移动到这个代表点
- 结果：162.07 TFLOPs/GPU, 55.91 GB peak

图 5：完整 logits 的估算公式、两种 TP/CP 切分及采用 Linear CE 后的代表性配置。来源：演讲 PPT，第 12 页。

页面给出两个算例：

配置	local_tokens	vocab_partition	完整 fp32 logits
TP=2, CP=2	65,536	124,160	约 32.55 GB
TP=1, CP=4	32,768	248,320	约 32.55 GB

第一种配置：

65,536 × 124,160 × 4 ≈ 32.55 GB

第二种配置：

32,768 × 248,320 × 4 ≈ 32.55 GB

当 TP 与 CP 的切分份额在 token 维和词表维之间重新分配时，一个局部维度缩小，另一个局部维度可能同步放大，二者乘积未必下降。

Linear CE（Linear Cross Entropy，线性交叉熵）针对的正是这个峰值。它将线性输出与交叉熵计算组织起来，避免生成并保留完整 logits。其意义不是先分配约 32.55 GB 再释放，而是不让这一完整张量成为必要的中间状态。

不能把 32.55 GB 直接从训练峰值里相减。logits、activation、临时 buffer 与通信张量的生命周期可能部分重叠，整体峰值取决于实际时序。材料也没有提供同配置、仅关闭 Linear CE 的对照实验。

采用 Linear CE 后，页面给出的代表性布局为：

- TP=1;
- PP=2;
- EP=8;
- CP=4;
- full recompute;
- 162.07 TFLOPS/GPU;
- 55.91 GB 峰值显存。

由于并行配置也发生变化，162.07 TFLOPS/GPU 不能归因于 Linear CE 单项改动。可靠结论是：Linear CE 解决完整 logits 的物化问题；如果某种实现本来就采用分块、融合或其他非完整物化路径，则不能照搬这项显存估算。

FSDP2：削减长期驻留的静态状态

去掉 loss 侧大张量后，参数、梯度和优化器状态仍会长期存在。这部分 static state 不能通过 activation 重计算消除，其占用取决于状态副本和分片方式。

为什么要看下面这张图？因为它比较了 distributed optimizer 与 FSDP2 的分片职责，并给出同一所列配置下的显存差异。



图 6：distributed optimizer 与 FSDP2 的状态职责及显存对比。来源：演讲 PPT，第 13 页。

页面将两种方案概括为：

方案	优化器状态	参数与梯度	页面类比
distributed optimizer	分片	仍受 PP/EP 拓扑约束并保留相应冗余	近似 ZeRO-1
FSDP2	全局分片	同样进行全局分片	近似 ZeRO-3

“近似 ZeRO-1”和“近似 ZeRO-3”只是帮助理解分片层级，不表示相关实现与标准 ZeRO 方案严格等价。

在 `TP=1■PP=2■EP=8■CP=4■full recompute` 的所列配置下，页面报告峰值显存从 55.91 GB 降至 47.03 GB，绝对下降：

55.91-47.03=8.88 GB

该页没有提供同配置吞吐、通信量或完整测量口径。因此，8.88 GB 可以作为静态显存下降的证据，不能进一步推出端到端性能收益。

FSDP2 更重要的作用是重新划分并行维度的职责：

维度	在本案例中的主要职责
CP	调节 local seq，在容量与计算粒度之间取平衡
EP	调节 MoE 计算与通信
PP	作为峰值显存或极限性能的可选细调项
TP	默认退出 recipe，存在明确模型或硬件理由时再启用

这不是所有模型和集群的通用规则。在更大规模下，FSDP2 的通信范围本身也可能成为约束。

至此，`full recompute` 处理 `activation`，`Linear CE` 处理完整 `logits`，FSDP2 处理静态状态。系统不再主要围绕 `OOM` 调整后，关键路径自然转向 `MoE dispatch/combine` 中暴露的 `all-to-all`。

五、从串行 all-to-all 到 chunked EP：重叠窗口如何出现

本节要回答：在 `full recompute` 路径下，怎样重新建立 MoE 通信与 expert compute 的重叠？

一次典型 MoE 前向包含：

- 1 `dispatch`：通过 `all-to-all` 将 token 发送到 expert 所在 rank；
- 2 `grouped GEMM`：执行专家分组矩阵乘法；
- 3 `combine`：通过 `all-to-all` 将 expert 输出送回原 rank。

FSDP2 不会自动隐藏这两次通信。旧的 `1F1B overlap` 又依赖不同 `microbatch` 或流水阶段之间的交错，演讲认为它不适配当前 `full recompute` 路径。

问题因此变为：不消除 `all-to-all`，如何重新创造可供通信和计算交错推进的窗口？

token 分块提供调度自由度

为什么要看下面这张图？因为它展示了 `chunked` `EP` 的最小机制：每块内部的依赖没有改变，新增的是跨块交错的可能性。

no-chunk vs chunk2

no-chunk

```
dispatch
→ grouped GEMM
→ combine
```

一个 MoE layer 一次性 dispatch、一次性 compute、一次性 combine。

chunk2

```
c0 dispatch → c0 GEMM → c0 combine
c1 dispatch → c1 GEMM → c1 combine
```

token 维切两块，通信流和计算流错位执行。

图 7：沿 token 维将一个 MoE 层拆为两个 chunk。来源：演讲 PPT，第 16 页。

no-chunk 路径为：

```
dispatch(all tokens)
→ grouped GEMM(all tokens)
→ combine(all tokens)
```

三步存在直接数据依赖。GEMM 必须等待 dispatch, combine 必须等待 GEMM。即使通信和计算使用不同 CUDA stream，单个大批次内部也缺少可独立推进的工作。

chunk2 沿 token 维拆出 **c0** 和 **c1**：

```
c0 dispatch → c0 GEMM → c0 combine
c1 dispatch → c1 GEMM → c1 combine
```

每个 chunk 内部仍遵循相同依赖，但两块之间可以错位：

- **c0** 进入 grouped GEMM 后，通信流有机会推进 **c1 dispatch**；
- **c1** 进入计算时，通信流又可以推进已经满足依赖的 combine；
- 一部分 all-to-all 因而进入另一块 expert compute 的时间窗口。

其机制可以压缩为：

token 分块

- 单次通信和计算粒度减小
- 一块计算期间出现另一块可调度的通信
- 通信流与计算流错位推进
- 部分 all-to-all 不再完全暴露。

图中的两条逻辑链并不表示 **c0** 与 **c1** 从头到尾完全并行。每块 GEMM 仍需等待本块 dispatch, combine 也仍需等待本块 GEMM。分块还会增加调度、同步和算子调用，材料没有量化这些固定成本。

前向 trace：关键窗口从约 10.06 ms 缩至 7.81 ms

为什么要看下面这张 trace？因为它把抽象的“双流重叠”落实到了实际时间轴。

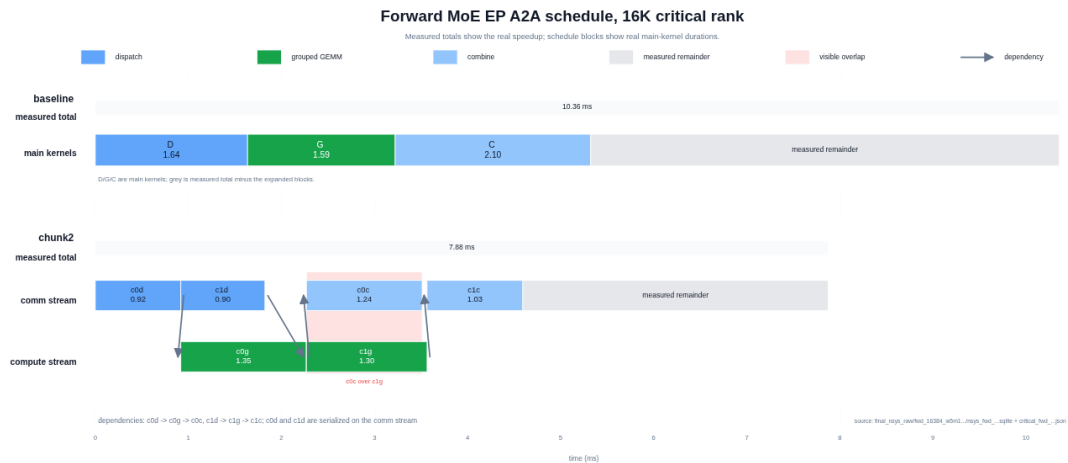


图 8：16K、critical rank 条件下的 Forward MoE EP all-to-all 调度对比。来源：演讲 PPT，第 17 页。

图中通信流负责 all-to-all，计算流负责 grouped GEMM。跨 stream waiting 箭头表示数据依赖：箭头到达之前，后继操作不能开始。CUDA 区域表示操作仍在等待依赖或可执行条件，不能视作有效计算。

baseline 中，dispatch、grouped GEMM 和 combine 基本串行排列，measured total 约为 **10.06 ms**。

chunk2 中，两块 grouped GEMM 与多段通信错位推进，measured total 约为 **7.81 ms**。该观测窗口缩短约：

$$10.06 - 7.81 = 2.25 \text{ ms}$$

这证明在该 critical rank 的前向窗口内出现了有效重叠，但不表示 all-to-all 已完全隐藏。图中仍存在等待区，而且这个结果不是所有 rank 的平均值，也不是完整训练 step 的耗时。

反向 trace：分块 dgrad，延后 wgrad

为什么要看下面这张图？因为反向不仅要安排通信，还要决定 dgrad 与 wgrad 何时占用计算流。

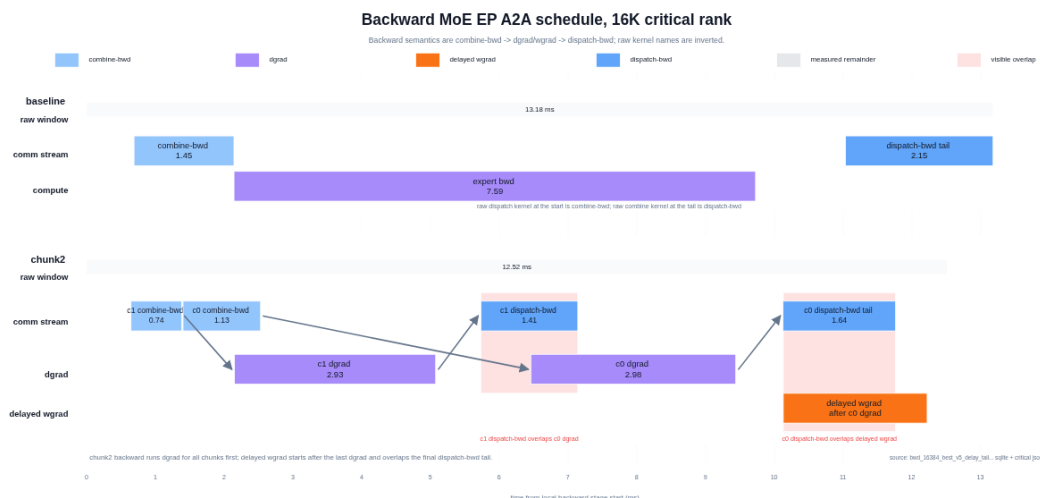


图 9：16K、critical rank 条件下的 Backward MoE EP all-to-all 调度对比。来源：演讲 PPT，第 18 页。

这里：

- **dgrad** 是对输入数据或 activation 的梯度计算；
- **wgrad** 是权重梯度计算。

baseline 的 raw window 约为 **13.83 ms**。

chunk2 将 dgrad 按 chunk 推进，使一块 dgrad 与另一块已经满足依赖的反向 all-to-all 交错执行。wgrad 则被移动到 delayed wgrad 区域，在关键通信与 dgrad 路径推进之后再执行，避免过早占用计算流。

chunk2 的 raw window 约为 **10.81 ms**，窗口缩短约：

$$13.83 - 10.81 = 3.02 \text{ ms}$$

但 raw window 不一定覆盖所有反向工作完成的时刻。延后的 wgrad 仍然需要执行，因此 3.02 ms 不能直接解释为完整反向或完整训练迭代的节省量。

融合重计算前向与反向仍是理想调度

演讲还展示了一条明确标为 **ideal** 的时间线：把重计算前向与反向组织成连续调度，让前向 grouped GEMM 后直接衔接分块 dgrad，反向通信在依赖满足后进入通信流，wgrad 继续延后。

这条时间线说明 chunk 边界还可能跨越“重计算前向 - 反向”阶段，从而扩大重叠窗口。但它是目标机制，不是实测端到端结果。材料没有给出这套融合调度的完整时延、吞吐提升或训练 step 加速比。

因此，chunked EP 的实质是以 token 分块换取调度自由度。收益取决于 local seq、专家负载均衡、通信与 GEMM 的相对时长，以及额外 launch 和同步成本。

六、证据如何读：单层趋势、整条路径与归因边界

前后向 trace 只能证明机制在 16K critical rank
的局部窗口内成立。要判断收益是否随序列增长，还需要看序列
sweep；要判断整条方案最终达到什么水平，则要看组合路径。

这三类证据的口径不同：

证据	能回答什么	不能回答什么
critical-rank trace	关键 rank 的局部时序是否形成重叠	所有 rank 平均时延、完整训练 step 加速
单层代理实验	单个 sparse MoE layer 随序列增长的趋势	完整模型显存和端到端吞吐
组合路径汇总	整套方案是否把运行点推进到更高吞吐	每项技术的独立加速比

单层代理实验：长 local seq 下观察到更大相对收益

实验对象是 Qwen3.5-35B-A3B 中的一个稀疏 MoE 层，只测 forward+backward，不包含 attention 和其他 Transformer 层。

为什么要看下面这张图？因为这里的重点不是某个单点，而是 baseline 与 optimized 两条显存曲线的斜率差异。

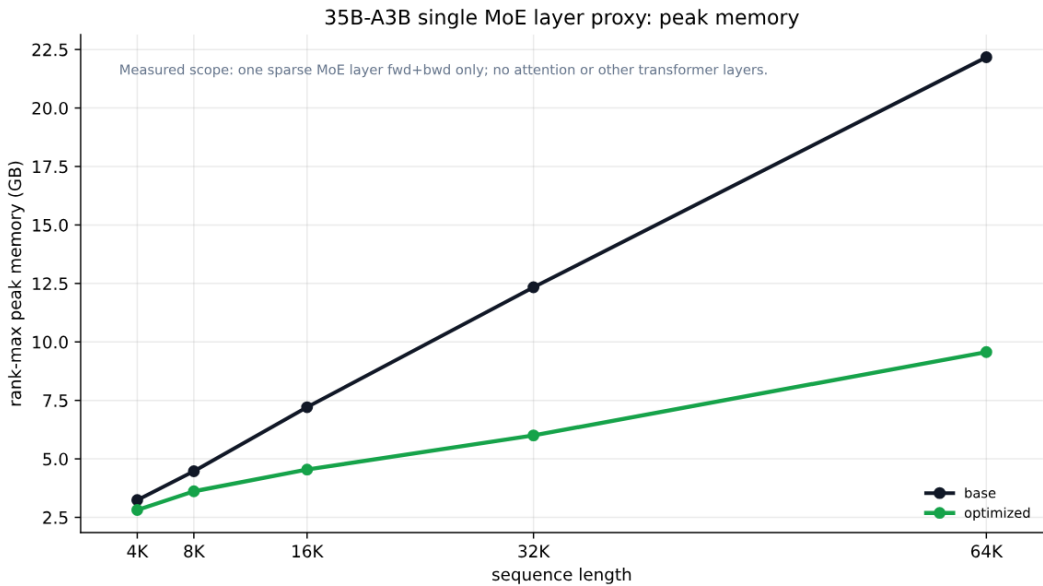


图 10：单个 sparse MoE layer 在不同 local sequence length 下的 rank-max 峰值显存。来源：演讲 PPT，第 20 页。

纵轴采用 rank-max peak memory：先记录各 rank
的峰值显存，再取最大值。它适合判断最吃紧的 rank 是否会阻止系统运行，但不等于所有 GPU
的平均显存。

横轴覆盖 4K、8K、16K、32K 和 64K local sequence length。两条曲线都随序列增长而上升，但优化后的斜率更低。64K 时，从图中近似读取：

- baseline 约 22.2 GB；
- optimized 约 9.6 GB。

这两个数字只能写作读图近似值，不能当作页面提供的精确测量。更稳妥的结论是：在该单层代理实验中，序列增长时两条显存曲线的差距扩大。

与之配套的单层 forward+backward 相对加速为：

local sequence length	相对 baseline 加速
4K	7.8%
8K	8.5%
16K	12.9%
32K	18.4%
64K	24.0%

这些数据表明，相对收益在所测范围内随 local seq 增长。一个可能的解释是：较长的本地计算区间能够摊薄固定调度与算子调用成本，并提供更多可供分块调度和通信重叠的工作。但现有序列 sweep 不能单独证明这一机制解释，也不能排除其他实现因素。

尤其需要注意，64K 下的 24.0% 只适用于这一个 sparse MoE layer 的 forward+backward。它不包含 attention，不能改写为完整训练 step 提速 24.0%。

组合路径：接近 190 TFLOPS/GPU 应怎样解释

为什么要看下面这张图？因为它同时展示了优化方法、并行配置、吞吐和峰值显存的变化。缺少其中任何一列，都容易做出错误归因。



图 11：吞吐、峰值显存与并行配置共同变化的优化路径。来源：演讲 PPT，第 22 页。

页面给出的代表性阶段为：

阶段	TP/PP/EP/CP	TFLOPS/GPU	峰值显存
baseline	2/4/8/2	127.53	42.91 GB
Linear CE	1/2/8/4	162.07	55.91 GB
FSDP2	1/2/8/4	163.06	47.03 GB
PP2→PP1	1/1/8/4	180.18	60.54 GB
chunked EP	1/1/8/4	185.96 - 187.10	37 - 38 GB

第一阶段到第二阶段不仅引入 Linear CE，还同时改变了 TP、PP 和 CP，因此 127.53 到 162.07 的差值不能全部归因于 Linear CE。

在所列的相同并行配置下，FSDP2 阶段从 162.07 变为 163.06 TFLOPS/GPU，更明确的变化是峰值显存从 55.91 GB 降至 47.03 GB。该对比没有给出完整的吞吐测量条件，因此不宜继续拆解二者之间的性能差异。

随后路径将 PP 从 2 改为 1，页面报告吞吐达到 180.18 TFLOPS/GPU，同时峰值显存升至 60.54 GB。由于这仍是组合路径中的阶段变化，现有材料不能把吞吐差值严格归因于减少 pipeline bubble，也不能断言 PP 调整完全由前一阶段的显存变化单独促成。

chunked EP 阶段继续保持 TP=1■PP=1■EP=8■CP=4，页面同时观察到更高吞吐和约 37 - 38 GB 峰值显存。结合前面的单层机制与 trace，可以确认分块为通信和计算重叠提供了调度粒度；但没有受控实验将最终峰值变化拆成 chunked EP 的独立贡献。

页面流程框把最终结果概括为约 **190 TFLOPS/GPU、37.09 GB**；表格则记录了更具体的 **185.96 - 187.10 TFLOPS/GPU、37 - 38 GB**。前者是近似概括，后者是页面中的代表范围，不能合并成“精确测得 190 TFLOPS/GPU”。

因此，这组结果能够支持：

组合方案把代表性吞吐从 127.53 TFLOPS/GPU 推进到接近 190 TFLOPS/GPU，并在最终阶段观察到约 37 - 38 GB 的峰值显存。

它不能支持：

Linear CE、FSDP2、降低 PP 或 chunked EP 分别贡献了多少独立加速或显存收益。

整条路径应当作为组合优化理解，而不是将相邻阶段的差值拆成单项技术收益。

七、让搜索空间收敛：从四维盲扫到约束驱动 recipe

确认组合路径有效后，下一个问题是如何避免继续穷举 PP、TP、EP、CP。

核心方法不是先找最优值，而是先给每个维度分配职责，再用显存和通信约束消去不必要的自由度。

为什么要看下面这张图？因为它把左侧的四维耦合搜索，压缩成右侧三个有顺序的配置决策。

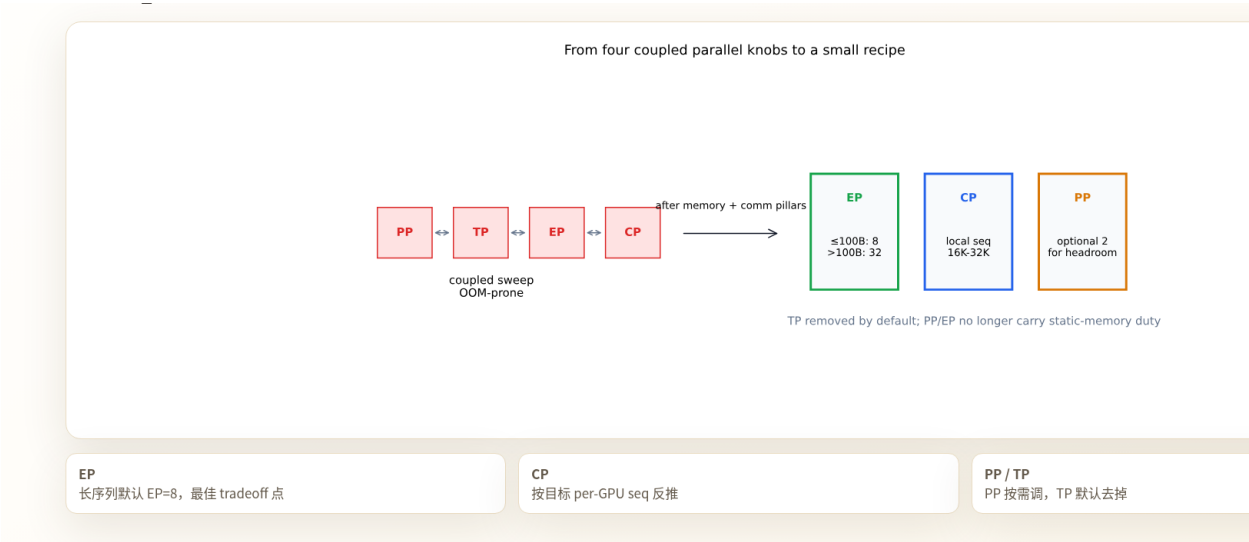


图 12：以显存与通信约束缩小并行配置空间。来源：演讲 PPT，第 23 页。

图中的箭头不是张量流，而是搜索空间的收敛过程：

- 1 先受 MoE 通信与拓扑约束，确定 EP；
- 2 再按目标 local seq 反推 CP；
- 3 最后判断是否需要 PP 提供额外显存余量；
- 4 TP 默认移出 recipe。

第一步：固定 EP 的默认起点

当前案例采用 EP=8 作为默认折中，而不是继续把 EP 当作自由搜索维度。

EP 增大可以减少每个 rank 持有的 expert 数量，却会改变 all-to-all 范围，并可能加重专家负载不均衡。它首先受到 MoE 通信与拓扑约束，而不是遵循“GPU 越多，EP 就应越大”的简单规则。

图中还出现了单位为 B 的 EP 阈值，但材料没有定义 B 的含义。正文不能把它换算为 token、batch size 或参数规模。

第二步：由目标 local seq 反推 CP

本案例希望把每张 GPU 的 local seq 控制在 16K - 32K。沿用演讲的简化关系：

$$S_local \approx (S_global) / (TP \times CP)$$

默认 TP=1 时，对 128K 全局序列：

CP	简化估算的 local seq	位置
4	32K	目标区间上界
8	16K	目标区间下界

因此，可以先把 CP 候选收敛到 4 - 8，而不是从大量组合开始盲扫。

这只是最小反推示例。实际映射仍受具体实现和其他并行维度影响，不能将该公式当作完整的显存与执行模型。

第三步：PP 只提供可选余量，TP 默认收起

在当前 recipe 中：

- PP=2 只在需要额外显存 headroom 时启用；
- TP 在没有明确模型或硬件理由时默认不使用；
- 若出现新的容量或拓扑约束，再重新引入 TP。

最终决策规则可以压缩为：

- 1 默认从 EP=8 开始；
- 2 按每卡 16K - 32K local seq 反推 CP；
- 3 显存余量不足时再评估 PP=2；
- 4 只有出现明确约束时才重新启用 TP。

这套并行配置依赖四项技术支柱：

技术组件	主要负责的资源问题
full recompute	以额外计算换取更大的 local seq
Linear CE	避免完整 fp32 logits 峰值
FSDP2	分片参数、梯度和优化器状态
chunked EP	处理暴露的 MoE all-to-all

四项之间是组合关系，不表示严格执行顺序。

这套方法只适用于当前模型、32 张 H100、128K 全局序列和演讲实现语境。它提供的是缩小搜索空间的方法，不是所有集群上的通用最优配置。MoE

八、把优化做成 primitive：缩小接入与验证半径

配置空间收敛后，工程改动仍可能跨越多条训练链路。本节要回答：如何让 FSDP2、Linear CE 和 chunked EP 以可局部开发、可成对验证的方式接入？

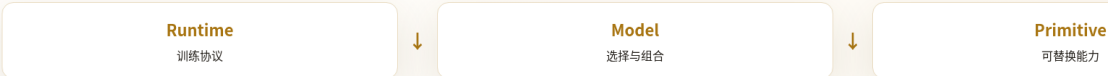
FSDP2 与多维并行组合时，会牵动进程组、参数聚合、梯度分片、优化器状态、checkpoint 和重计算。chunked EP 与 full recompute 组合时，则会改变 dispatch/combine 时序、前后向依赖、delayed wgrad 位置和 buffer 生命周期。

材料没有量化工期和代码改动量，但明确表明主要成本来自工程接入与验证，而不是底层 kernel 能否完成计算。

Runtime、Model、Primitive 三层组织

为什么要看下面这张图？因为它表达的是工程职责关系，而不是训练时的张量数据流。

Megatron-Lite：换组织形式，不换 kernel



模型由 primitive 组合出来，避免继承一个不断膨胀的大模型类。

与 Megatron-Core 复用相同的底层 kernel，性能和精度完全一致；改变的只是组织形式和改动边界。

图 13：Megatron-Lite 在复用 Megatron-Core 底层 kernel 的前提下重组系统。来源：演讲 PPT，第 26 页。

三层分别负责：

- **Runtime**：训练协议和运行过程如何推进；
- **Model**：连接并组合模型需要的能力；
- **Primitive**：提供可替换的 loss、状态分片或通信能力。

图中的箭头表示架构组织关系：Runtime 通过 Model 使用组合后的 primitive。它不是通信方向，也不表示显存或张量传输时序。

这种组织形成了一个更小的验证闭环：

- 明确能力边界
- 局部替换实现
 - 在固定口径下成对比较
 - 验证通过后重新组合
 - 缩小接入和回归范围。

材料没有给出三层的正式接口定义，因此不能进一步推断具体类结构或调用约束。

三项优化对应三类 primitive

优化	Primitive 边界	主要验证关注点
Linear CE	loss	loss 正确性与 logits 峰值
FSDP2	optimizer/state sharding	参数、梯度和优化器状态分片
chunked EP	MoE communication	all-to-all 是否与计算形成重叠

这不是运行顺序，而是功能边界映射。

Linear CE 回答“损失是否正确、完整 logits 峰值是否下降”；FSDP2 回答“状态是否按预期分片”；chunked EP 回答“dispatch/combine 时序是否形成有效重叠”。三者的输入、输出与测试指标不同，不能使用同一组性能数字相互替代。

Paired test：把正确性与性能分开验证

演讲提出的开发闭环包含四步：

- 1 **Skill**：读取 primitive 的说明，确认能力边界和验证口径；
- 2 **Change**：修改局部实现；
- 3 **Paired test**：让新旧实现成对运行；
- 4 **Compose**：测试通过后，将新 primitive 重新组合进模型。

paired test 检查四项指标：

- **loss**：损失是否对齐；
- **grad**：梯度是否对齐；
- **peak**：峰值显存是否符合预期；
- **time**：耗时是否回归或改善。

其中 **loss** 和 **grad** 面向数值正确性，**peak** 和 **time** 面向资源与性能。性能改善不能替代数值正确，数值通过也不表示性能目标已经达到。

材料没有提供误差容限、随机种子、对照基线、测试硬件、自动修改成功率或人工介入比例。因此，paired test 目前只能被理解为验证框架，不能据此宣称已经形成完备的自动验收标准。

演讲者还主张，Megatron-Lite 与 Megatron-Core 复用相同底层 kernel，因此性能和精度完全一致；转写进一步提出，减少部分 CPU 条件判断后，轻量路径可能更快。这些说法缺少同配置基准、误差范围和独立验证。kernel 相同并不足以自动推出端到端行为完全一致，“可能更快”也只能作为待验证判断。

Megatron-FSDP、训练用 mega kernel、multi-LoRA、QAT、更多模型支持，以及异步 rollout、weight resharding，均属于后续规划，不能当作已经交付并验证的能力。

结论与局限

这条优化路径的价值不在于简单堆叠技巧，而在于让每类资源压力拥有相对明确的处理机制。

- 1 **长序列 MoE RL 的首要问题是可运行性与试验效率**。在 32 张 H100 上支持 128K 全局序列，是为多实验并行和快速试错建立稳定起点，不是讨论无限扩展集群后的峰值性能。
- 2 **PP、TP、EP、CP 和重计算不能分别调优**。它们会共同改变静态显存、动态显存、kernel 粒度、暴露通信、CPU 开销和 local seq，单个可运行点不能提供单变量因果结论。
- 3 **full recompute 用额外计算交换更大的 local seq**。材料给出的代价约为 30% 额外计算量；它直接缓解的是 activation 显存，不等于训练时间必然增加 30%。

- 4 **Linear CE 与 FSDP2 分别处理两类不同峰值。** Linear CE 避免完整 fp32 logits 物化；FSDP2 分片参数、梯度和优化器状态，使静态显存不再主要依赖 PP、EP 承担。
- 5 **chunked EP 通过 token 分块创造通信重叠窗口。** 前向约 10.06→7.81 ms、反向 raw window 约 13.83→10.81 ms，只证明 16K critical rank 的局部时序改善，不能外推为完整训练 step 的平均加速。
- 6 **整条组合路径有效，但单项收益不可从阶段差值中拆出。** 代表性结果从 127.53 TFLOPS/GPU 推进到接近 190 TFLOPS/GPU；表格中的最终范围是 185.96 - 187.10 TFLOPS/GPU、37 - 38 GB。由于阶段间并行配置同时变化，不能据此计算各项技术的独立贡献。
- 7 **recipe 和 primitive 分别缩小运行搜索半径与工程验证半径。** 当前方案默认 EP=8，按每卡 16K - 32K local seq 反推 CP，按需使用 PP=2，并默认收起 TP；工程上再将 loss、状态分片和 MoE 通信拆成可成对测试的 primitive。

最后必须保留三项证据边界：显存与 speedup sweep 只覆盖单个 sparse MoE layer 的 forward+backward；前后向 trace 只覆盖 16K critical rank；最终 recipe 只适用于当前模型、32 张 H100 和演讲实现语境。它是一套约束驱动搜索方法，而不是所有长序列 MoE 训练任务的通用最优答案。